

Q.No:8

Question description

A company has n employees with certain salaries. Your task is to keep track of the salaries and process queries.

Constraints

- $1 \leq n, q \leq 2 \cdot 10^5$
- $1 \leq p_i \leq 10^9$
- $1 \leq k \leq n$
- $1 \leq x \leq 10^9$
- $1 \leq a \leq b \leq 10^9$

Input

The first input line contains two integers n and q : the number of employees and queries. The employees are numbered $1, 2, \dots, n$.

The next line has n integers $p_1, p_2, \dots, p_n$: each employee's salary.

After this, there are q lines describing the queries. Each line has one of the following forms:

- $! k x$: change the salary of employee k to x
- $? a b$: count the number of employees whose salary is between $a \dots b$

Output

Print the answer to each $?$ query.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 3
3 7 2 2 5
? 2 3
! 3 6
? 2 3
```

EXPECTED OUTPUT

```
3
2
```

Test Case 2

INPUT (STDIN)

```
5 3
31 17 12 12 15
? 12 31
! 13 6
? 2 13
```

EXPECTED OUTPUT

```
5
3
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int compare(const void
*a, const void *b)
```

Test Case 2

KEYWORD

```
void update(int i, int
n, int x)
```

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 200005

int salary[MAX];
int values[MAX];
int bit[MAX];

int compare(const void *a, const void *b)
{
    int x = *(int *)a;
    int y = *(int *)b;

    if(x < y) return -1;
    if(x > y) return 1;
    return 0;
}

void update(int i, int n, int x)
{
    while(i <= n)
    {
        bit[i] += x;
        i += i & -i;
    }
}

int query(int i)
{
    int sum = 0;

    while(i > 0)
    {
        sum += bit[i];
        i -= i & -i;
    }

    return sum;
}

int lowerBound(int arr[], int n, int x)
{
    int l = 0;
    int r = n;

    while(l < r)
    {
```

```
    int mid = (l + r) / 2;

    if(arr[mid] < x)
        l = mid + 1;
    else
        r = mid;
}

return l;
}

int upperBound(int arr[], int n, int x)
{
    int l = 0;
    int r = n;

    while(l < r)
    {
        int mid = (l + r) / 2;

        if(arr[mid] <= x)
            l = mid + 1;
        else
            r = mid;
    }

    return l;
}

int main()
{
    int n, q;
    int originalN;
    int i;

    scanf("%d %d", &n, &q);

    originalN = n;
    for(i = 0; i < n; i++)
    {
        scanf("%d", &salary[i]);
        values[i] = salary[i];
    }
    char type[MAX];
    int x[MAX], y[MAX];

    int total = n;

    for(i = 0; i < q; i++)
    {
        scanf(" %c %d %d", &type[i], &x[i], &y[i]);
```

```
    if(type[i] == '!')
    {
        values[total] = y[i];
        total++;
    }
}
qsort(values, total, sizeof(int), compare);
int m = 0;

for(i = 0; i < total; i++)
{
    if(i == 0 || values[i] != values[i - 1])
    {
        values[m] = values[i];
        m++;
    }
}
printf("Output:\n");
for(i = 0; i < originalN; i++)
{
    int pos = lowerBound(values, m, salary[i] + 1;

    update(pos, m, 1);
}
for(i = 0; i < q; i++)
{
    if(type[i] == '!')
    {

        int oldPos =
            lowerBound(values, m, salary[x[i] - 1]) + 1;

        int newPos =
            lowerBound(values, m, y[i]) + 1;

        update(oldPos, m, -1);
        update(newPos, m, 1);

        salary[x[i] - 1] = y[i];
    }
    else
    {

        int left =
            lowerBound(values, m, x[i]);

        int right =
            upperBound(values, m, y[i]);

        int answer =
            query(right) - query(left);
    }
}
```

```
    printf("%d\n", answer);  
}  
}  
  
return 0;  
}
```

Output:

Test_case 1:

```
5 3  
3 7 2 2 5  
? 2 3  
! 3 6  
? 2 3  
Output:  
3  
2
```

Test_case 2:

```
5 3  
31 17 12 12 15  
? 12 31  
! 13 6  
? 2 13  
Output:  
5  
2
```